Lab Report

CYBERSECURITY OPERATIONS

Done by VAINQUEUR IGIRANEZA

Table of Contents

1. Environment and Network Configuration 3

2. Onboarding method (agent vs syslog) and data sources in SIEM 3

3. Attack Selection and Rationale 4

4. Mapping to Cyber Kill Chain and MITRE ATT&CK 4

4.1 Discovering the vulnerable web endpoint Cyber Kill Chain stage: Reconnaissance 5

Evidence and telemetry: 5

Mitigations: 5

Evidence and telemetry: 6

4.3 Actions on objectives, unauthorized data access, and confidentiality impact Cyber Kill Chain stage: Actions on Objectives 6

Evidence and telemetry: 6

Mitigations: 6

4.4 Summary of framework alignment 7

5. Detection logic & rule(s) screenshots & plain-English description 7

6. Tuning iterations (what failed, what changed, final result) 8

7. Ethics & safety note (how did you preserve scope & ensure isolation) 8

References 9

Appendix A: Screenshots 10

1. Environment and Network Configuration

The lab environment was deployed using VirtualBox and configured with a host-only network to ensure full isolation from external networks. This setup guarantees that all attack and detection activities remain within the local lab scope and comply with ethical and course requirements. The environment consists of three virtual machines:

Table 1: Virtual machines:

VM Role	Operating System	IP Address	Description
Attacker	Kali Linux	192.168.56.101	Used to perform the SQL Injection attack
Target	OWASP Juice Shop	192.168.56.109	Intentionally vulnerable web application
SIEM	Wazuh	192.168.56.108	Centralized log collection, correlation, and alerting

2. Onboarding method (agent vs syslog) and data sources in SIEM

To enable centralized detection and monitoring, the OWASP Juice Shop target system was onboarded into the SIEM using a Wazuh agent-based integration. The agent was installed and configured on the target virtual machine to securely forward system and application logs to the Wazuh manager. Once the agent was connected, telemetry from the target system became visible in the Wazuh dashboard, including operating system events and web application-related logs. This confirmed that the SIEM was correctly ingesting data prior to executing the attack (Hernandez, 2023).

3. Attack Selection and Rationale

The selected attack for this lab was a SQL injection attack against the OWASP Juice Shop login endpoint conducted within an isolated VirtualBox host-only environment. The attack was chosen based on its relevance, impact, and suitability for both offensive execution and defensive detection within a SIEM (D. Stuttard & M. Pinto, 2023).

The lab environment consists of Kali Linux as the attacker, OWASP Juice Shop as the vulnerable target, and Wazuh as the SIEM, all connected through a host-only network to ensure ethical and safe testing. The SQL injection attack targets improper input validation in the authentication mechanism of the application by using crafted login payloads to bypass authentication controls. From a MITRE ATT&CK perspective, this activity maps to the Initial Access tactic, as the attacker is attempting to gain unauthorized entry into the target application (Corporation, 2024).

The specific technique associated with this behavior is T1190 – Exploit Public-Facing Application, since the attack exploits a vulnerability in a publicly accessible web application endpoint. SQL injection is a common exploitation method used under this technique to manipulate backend database queries and bypass security controls (Miller, 2021).

4. Mapping to Cyber Kill Chain and MITRE ATT&CK

This section maps the executed SQL injection attack against OWASP Juice Shop to both the Cyber Kill Chain and the MITRE ATT&CK framework. The goal is to show how a single, focused web application attack traverse's multiple offensive stages and how each of these stages can be understood, detected, and mitigated using standardized security taxonomies (Seitz, 2022).

Table 2: Overview mapping

Cyber Kill Chain stage	**MITRE ATT&CK tactic**	**Technique ID**	**Technique name**	**High-level procedure in this lab**
Reconnaissance	Reconnaissance	—-	—	Identifying the exposed Juice Shop endpoint.
Delivery	Initial Access	T1190	Exploit Public-Facing Application.	Sending SQL injection payload via HTTP request. Exploit of the /rest/products/search endpoint via SQLi.
Exploitation	Initial Access	T1190	Exploit Public-Facing Application	Exploiting vulnerable input validation in the login/search
Actions on Objectives	Collection	—	—	Retrieving data and accessing user profile without authorization

4.1 Discovering the vulnerable web endpoint

Cyber Kill Chain stage: Reconnaissance

ATT&CK tactic: Reconnaissance

The attacker used the Kali Linux browser to interact with the OWASP Juice Shop interface and identify potential injection points. This included examining how user input was processed, observing parameterized requests, and determining which endpoints accepted externally supplied data. This behavior reflects typical web application enumeration, where an adversary surveys the application to identify weaknesses suitable for exploitation.

Evidence and telemetry:

This phase primarily produced browser interaction and shows an early request to log in using a normal email and password from the attacker's IP (192.168.56.101) to the Juice Shop server, which is visible in the Nginx access logs as shown in (Figure 1), showing normal requests before the malicious payload. These baseline entries help distinguish regular browsing from the later attack pattern. (Lopez, 2021)

```
a/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" ""
192.168.56.101 - - [20/Feb/2026:09:53:43 +0000] "GET /rest/user/whoami HTTP/1.1" 200 11 "http://192.168.56.109/" "Mozill
a/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" ""
192.168.56.101 - - [20/Feb/2026:09:53:43 +0000] "POST /rest/user/login HTTP/1.1" 401 26 "http://192.168.56.109/" "Mozill
a/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" "{\x22email\x22:\x22admin@admin.com\x22,\x22password\x
22:\x22****\x22}"
192.168.56.101 - - [20/Feb/2026:09:53:45 +0000] "GET /socket.io/?EIO=4&transport=polling&t=PnxLc6v&sid=z3FJIWfvfhdqw39yA
AAh HTTP/1.1" 200 1 "http://192.168.56.109/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" ""
192.168.56.101 - - [20/Feb/2026:09:53:45 +0000] "POST /socket.io/?EIO=4&transport=polling&t=PnxLi24&sid=z3FJIWfvfhdqw39y
AAAh HTTP/1.1" 200 2 "http://192.168.56.109/" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" "
3"
```

Figure 1: Normal injection log

Mitigations:

- **Attack surface reduction:** Limit the number of exposed endpoints and disable unused functionality.
- **Application hardening:** Implement robust access control even for seemingly benign features such as search, reducing the risk that reconnaissance leads to exploitable inputs.
- **Logging of reconnaissance patterns:** Monitor unusual enumeration behavior (e.g., repeated parameter tampering, abnormal error responses) to detect early probing.

4.2 Initial Access – exploiting the public-facing application (T1190)

Cyber Kill Chain stages: Delivery and Exploitation

ATT&CK tactic: Initial Access

ATT&CK technique: T1190 Exploit Public-Facing Application

Procedure in the lab: The main offensive action consisted of sending an SQL injection to the Juice Shop login with a SQL injection payload in the query parameter. The payload was: ' OR 1=1 --.

This input manipulated the backend SQL query, causing the application to treat the condition as always true and return data without enforcing correct authentication.

According to MITRE ATT&CK, this behavior falls under Exploit Public-Facing Application (T1190), where an adversary abuses a vulnerability in an Internet- or network-facing application to gain unauthorized access.

Evidence and telemetry:

- **Web server logs:** The Nginx access log on the Juice Shop VM records requests from 192.168.56.101 containing the details of sql injection string in appendix A (Figure 4).

4.3 Actions on objectives, unauthorized data access, and confidentiality impact

Cyber Kill Chain stage: Actions on Objectives

ATT&CK tactic: Collection (with elements related to Account Access / Abuse)

Procedure in the lab: After exploiting the injection point, the attacker can login the the admin profile without proper authentication. In a related step, the attacker was also able to access a user profile page in the Juice Shop interface without valid login credentials. Together, these actions demonstrate that exploitation progressed beyond simple vulnerability confirmation and resulted in the achievement of a real objective: unauthorized access to application data and user functionality as shown in (Figure 6).

Evidence and telemetry:

- **HTTP response content:** The browser on Kali shows a successful login attempt after the injection payload was submitted. This confirms that the backend executed a broader query than intended.
- **Application state:** The user profile page became accessible from the attacker's session without a legitimate login, indicating that the authentication mechanism was bypassed or weakened, and that the attacker gained access to sensitive functionality.
- **SIEM view:** From a defensive perspective, these events appear in Wazuh as a part of the same time window as the SQL injection alert, helping analysts reconstruct the timeline from malicious input to sensitive data access.

Mitigations:

- **Stronger authorization checks:** Enforce strict server-side access control on all sensitive resources, ensuring that data and profile endpoints require valid authentication and appropriate roles, regardless of how the request is formed.
- **Fine-grained logging of access to sensitive objects:** Record successful and failed access attempts to user profiles and other sensitive resources, including source IP and user identity, and forward these logs to the SIEM for correlation.
- **Data minimization and response hardening:** Avoid returning unnecessary data fields in responses, especially to unauthenticated or low-privilege requests. Apply output filtering and error handling so that application behavior does not leak excessive information to an attacker.

4.4 Summary of framework alignment

By combining the Cyber Kill Chain and MITRE ATT&CK perspectives, the SQL injection attack in this lab can be seen as more than a single exploit. It follows a structured progression from reconnaissance, through exploitation of a public-facing application (T1190) in appendix A (Figure 3), to concrete actions on objectives in the form of unauthorized data access and profile viewing. The Wazuh SIEM's ability to detect the injection attempt and tie it to host-based telemetry demonstrates how such frameworks support not only offensive analysis but also defensive engineering, tuning, and continuous improvement of detection capabilities.

5. Detection logic & rule(s) screenshots & plain-English description

An SQL injection payload was used on the OWASP Juice Shop admin login, which allowed the backend database to process the request and bypass authentication. This resulted in a confidentiality breach because the attacker accessed the system without valid credentials as shown in (Figure 2) below.

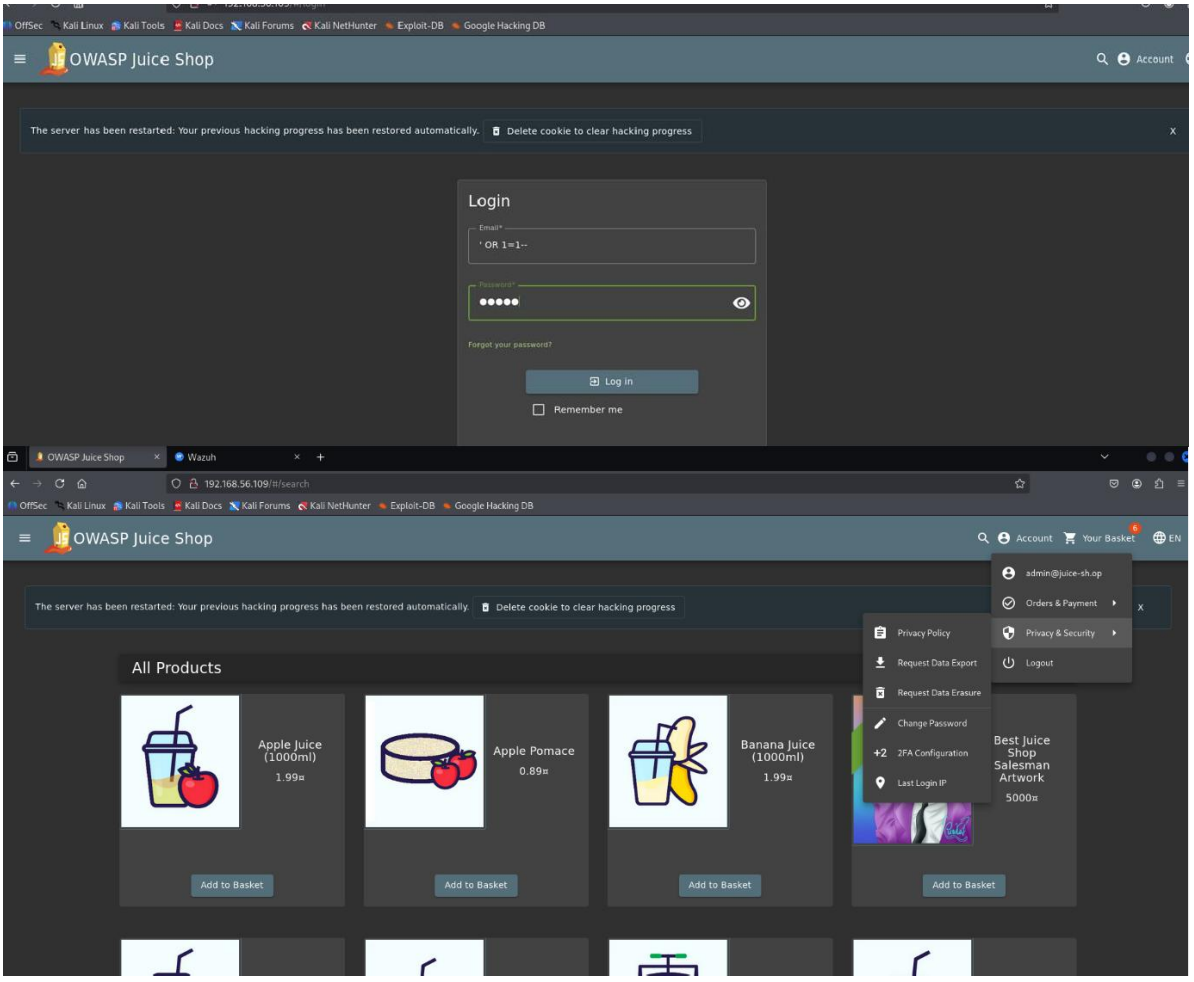


Figure 2: SQL injection in JS login

We get the Nginx access logs for OWASP Juice Shop, which showed HTTP GET requests with the SQL injection payload (' OR 1=1--) from the attacker's virtual machine at IP address 192.168.56.101, providing clear server-side evidence of the attack in Appendix A (Figure 4).

Wazuh SIEM detected the SQL injection attempt by examining web server logs from the

monitored target host (JUICESHOP-VM). The system correctly identified the bad request and sent out an alert saying, "SQL injection attempt" as shown in (Figure 5).

Appendix A (Figure 5) shows the details of the SQL injection attempt shown in the wazuh event for loging in juice shop.

We were able to access a user profile in the OWASP Juice Shop app without proper authentication after carrying out the SQL injection attack. This proves that the attack got around authentication controls and let someone who shouldn't have access to sensitive user functions, which is a breach of privacy. Proof is provided in appendix A (Figure 6).

6. Tuning iterations (what failed, what changed, final result)

At first, the SQL injection attack did not trigger any alerts in Wazuh, even though the payload (' OR 1=1--) reached the Juice Shop login endpoint (even when it can be seen in the nginx log file) and caused a web server 400 error code. A custom Wazuh detection rule was created to find SQL injection patterns in Nginx access logs. Uploading the rule through the Wazuh Dashboard showed a successful SQL injection attempt. When we check the details of the event, it showed the false positive as shown in (Figure 7).

This meant that there was a problem with the rule, as it was a very general rule and returned a false positive instead of showing a successful attempt of SQL injection ('OR 1=1 --) in the login section of juiceshop. Therefore, we have to change the rule again and make it specific to detect the right and positive attempt after SQL injection in JuiceShop. After these changes, the rule was put into use by restarting the Wazuh manager, and it was able to reliably find SQL injection attempts. The custom rule we applied can be seen in (Figure 8).

7. Ethics & safety note (how did you preserve scope & ensure isolation)

All activities were done in a controlled and separate lab setting that followed ethical cybersecurity rules. The lab had a VirtualBox host-only network, which meant that all attack traffic, logs, and telemetry stayed in the local environment and were not connected to external systems or the public internet. We chose OWASP Juice Shop as the target app because it is purposefully weak and made for security training and testing.

The SQL injection attack was performed only to demonstrate how to bypass authentication and access data without permission for defensive detection and analysis. No destructive actions, persistence, denial-of-service, or system changes occurred. All the virtual machines were owned and run by the people taking the course, and there were no real user data or production systems involved. We only used Wazuh SIEM for defensive monitoring, adjusting detection rules, and checking security controls. Because of a clear scope, strict isolation, and the use of software that was meant to be vulnerable, all activities were safe, responsible, and met the learning goals.

References

Hernandez, J. (2023). Practical purple teaming: Strengthening offensive and defensive collaboration. O'Reilly Media, Inc.

Lopez, J. (2021). Mastering defensive security: Effective techniques to secure your enterprise. O'Reilly Media, Inc.

Miller, C. (2021). Cybersecurity blue team toolkit. John Wiley & Sons, Inc.

MITRE Corporation. (2024). MITRE ATT&CK® framework. MITRE Corporation.

Stuttard, D., & Pinto, M. (2023). Web application security: Exploitation and defense (2nd ed.). O'Reilly Media, Inc.

Seitz, B. (2022). Practical purple teaming: Adversary emulation and detection engineering. O'Reilly Media, Inc.

Appendix A: Screenshots

This appendix contains the addition screenshots referenced in the main text.

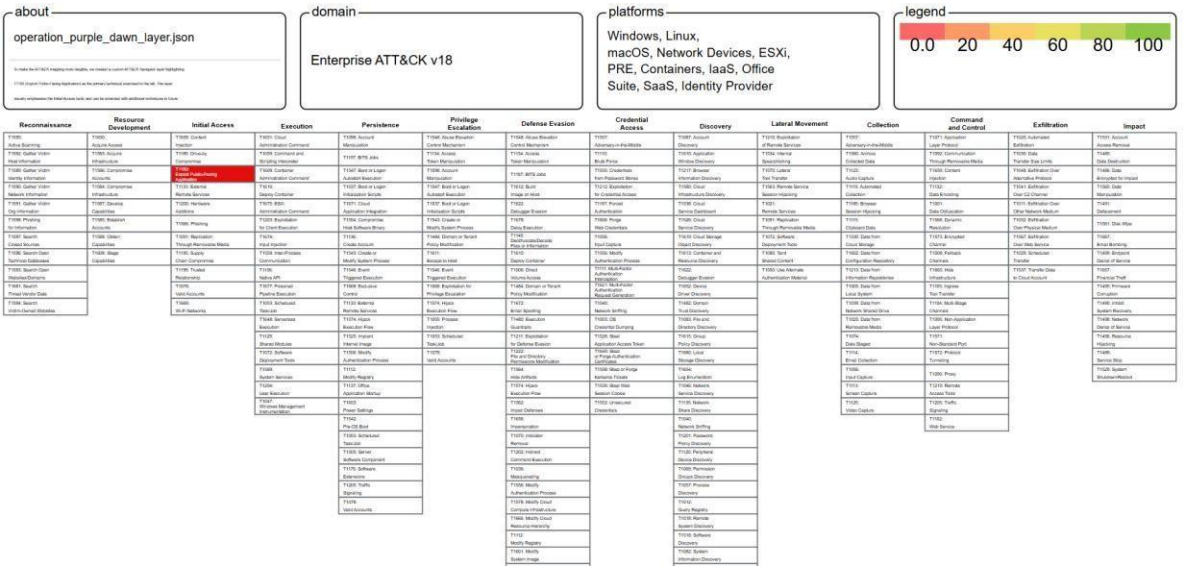
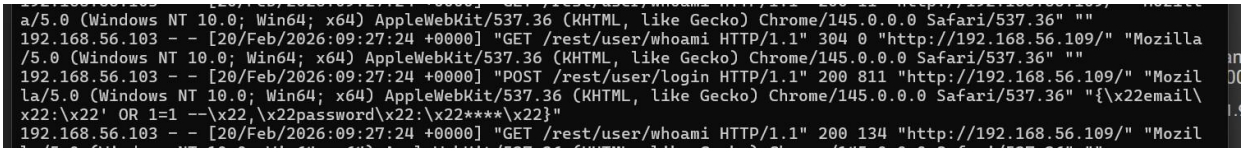


Figure 3: MITRE ATT&CK Navigator



```
a/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" ""
192.168.56.103 - - [20/Feb/2026:09:27:24 +0000] "GET /rest/user/whoami HTTP/1.1" 304 0 "http://192.168.56.109/" "Mozilla
/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" ""
192.168.56.103 - - [20/Feb/2026:09:27:24 +0000] "POST /rest/user/login HTTP/1.1" 200 811 "http://192.168.56.109/" "Mozil
la/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36" "{\x22email\
x22:\x22' OR 1=1 --\x22,\x22password\x22:\x22****\x22}"
192.168.56.103 - - [20/Feb/2026:09:27:24 +0000] "GET /rest/user/whoami HTTP/1.1" 200 134 "http://192.168.56.109/" "Mozil
```

Figure 4: Nginx access logs showing SQL injection payload

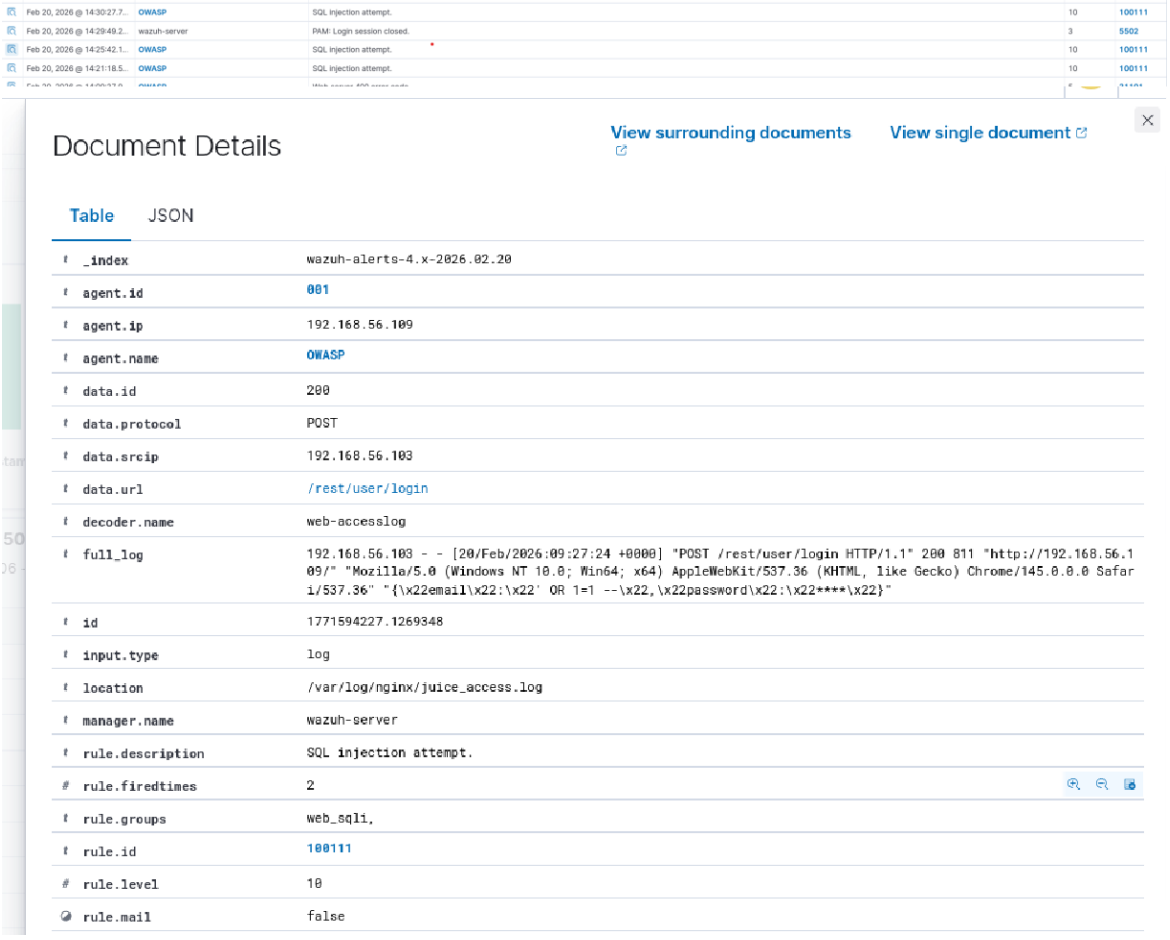


Figure 5: SQL Injection attempt and event details

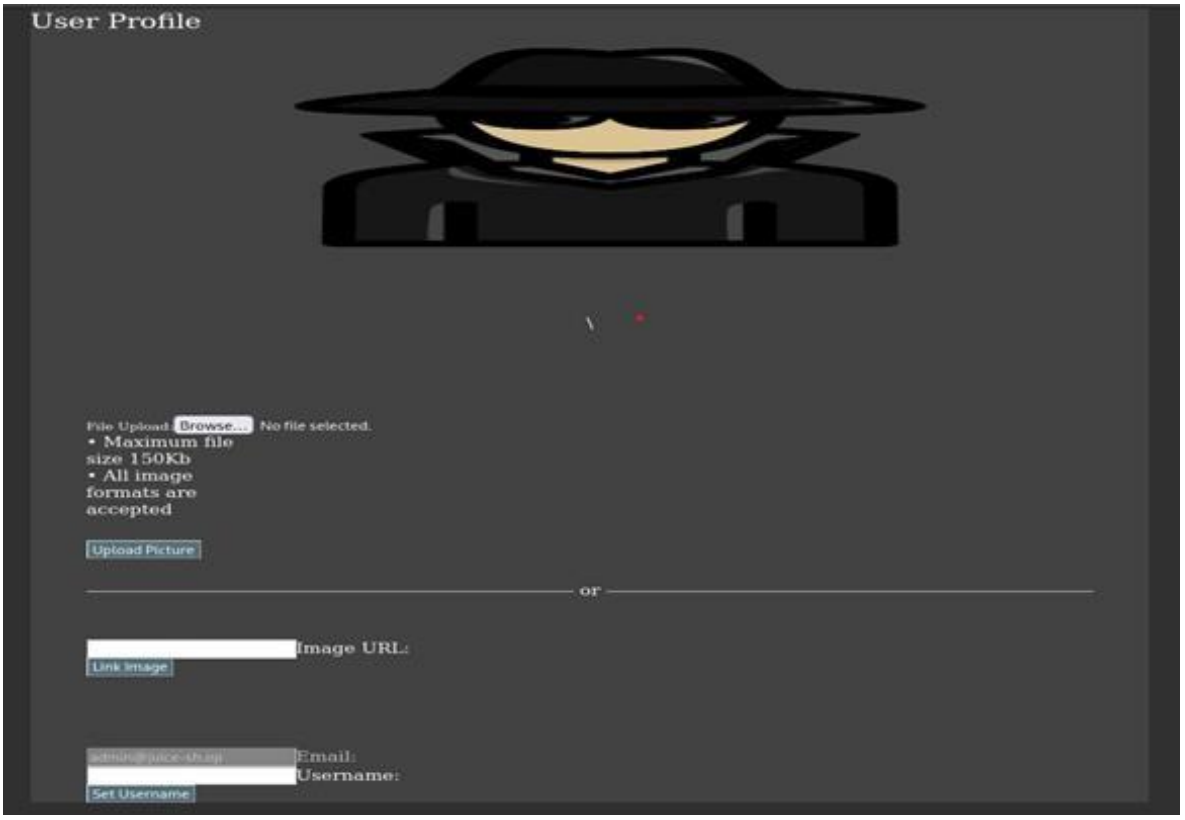


Figure 6: Unauthorized Access to user profile after SQL injection

Document Details

View surrounding documents

View single document

Table JSON

_index	wazuh-alerts-4.x-2026.01.16
agent.id	001
agent.ip	192.168.56.109
agent.name	OWASP
data.id	400
data.protocol	GET
data.srcip	192.168.56.101
data.url	/socket.io/?EIO=4&transport=websocket&sid=kBhaV3jD2Ac32ufBAAAK
decoder.name	web-accesslog
full_log	192.168.56.101 - - [16/Jan/2026:05:38:20 +0000] "GET /socket.io/?EIO=4&transport=websocket&sid=kBhaV3jD2Ac32ufBAAAK HTTP/1.1" 400 45 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0" ""
id	1768566684.1206161
input.type	log
location	/var/log/nginx/juice_access.log
manager.name	wazuh-server
rule.description	SQL Injection attempt detected in web request
rule.firedtimes	1
rule.groups	web, attack, sqli
rule.id	100201

Figure 7: False positive injection details

Wazuh | Wazuh | Error: Unexpected path: /rest | W

https://192.168.56.108/app/rules#/manager/?tab=ruleset

OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Goog

W. Rules

newrule.xml

```
<group name="web_sqli, ">
  <rule id="100111" level="10">
    <if_group>web</if_group>
    <url>/rest/user/login</url>
    <match>OR 1=1</match>
    <description>SQL injection attempt.</description>
  </rule>
</group>
```

Figure 8: Wazuh custom rule